Republic of the Philippines
CAMARINES SUR POLYTECHNIC COLLEGES
Nabua, Camarines Sur

COLLEGE *of* COMPUTER STUDIES

A Simple Calculator in Python Language with a Text Typing Corrector

A Final Project and Final Examination

Presented to the

College of Computer Studies

Camarines Sur Polytechnic Colleges

Nabua, Camarines Sur

In Fulfillment of the Requirements in Fundamentals of Programming (CCCS102)

Atienza, Lawrence

Doroin, Ivy M.

Orias, Mark Joseph C.

BSCS 1A

Mr. Ibo, Allan O. Jr.

CCCS102 Instructor

December 2023

DOCUMENTATION

I. INTRODUCTION.

The advent of technology has brought about significant changes in the way we live, work, and communicate. Python programming has emerged as one of the most versatile and widely used languages in computer science due to its simplicity and ease of use. This project aims to explore the fundamental concepts of Python programming. The project will delve into the basic input/output for user interaction, implementing conditional statements and looping structures, string manipulation, file handling, and exception handling.

The provided Python code is an example of a simple calculator application. This application is capable of performing basic arithmetic operations such as addition, subtraction, multiplication, and division. One of the unique features of this calculator is its ability to maintain a history of operations performed. This history can be saved to or loaded from a file, providing a useful record of calculations.

Additionally, it also includes a user-friendly menu system. This system allows users to choose an operation, input the numbers for the operation, and view the result. Users can also choose to save the history of operations to a file or load a history from a file.

In addition to the calculator functionality, the program includes a tool to correct certain characters in a text file. This tool replaces specific numbers with corresponding letters, following a predefined mapping. This could be useful for correcting files that have been encoded in a certain way.

II. PROJECT TASK.

Calculator:

A. TASK 1 ALGORITHM DESIGN AND PYTHON BASICS

As part of our initial project task, we have been assigned to design an algorithm that solves a specific problem and then code it into Python. The provided Python code is a simple calculator application with the ability to perform basic arithmetic operations, maintain a history of operations, and save or load this history from a file.

The code defines a ***Calculator*** class, which is a blueprint for creating calculator objects. Each calculator object has its own ***history*** list to keep track of operations. The class includes methods for addition, subtraction, multiplication, and division, as well as methods to save and load the history.

The code is executed in a top-down manner, starting from the first line to the last line. For example, in the ***add*** method, the sum of the two numbers is calculated first, then the operation is appended to the history, and finally the result is returned.

The code uses **basic input/output for user interaction**. It uses the **input function** to get the user's choice and the numbers for the operations, and the **print function** to display the menu and the results.

ALGORITHM (Pseudocode)

```
1. CREATE a class `Calculator` with the following methods:
    1. `__init__`: INITIALIZE an empty list `history`.
    2. `add`: ADD two numbers, APPEND the operation to
`history`, and RETURN the result.
    3. `subtract`: SUBTRACT two numbers, APPEND the operation
to `history`, and RETURN the result.
    4. `multiply`: MULTIPLY two numbers, APPEND the operation
to `history`, and RETURN the result.
    5. `divide`: IF the second number is zero, APPEND an error
message to `history` and RETURN None. ELSE, DIVIDE two numbers,
APPEND the operation to `history`, and RETURN the result.
    6. `save_history`: OPEN a file with the given filename in
write mode, WRITE each operation in `history` to the file.
```

```
    7. `load_history`: TRY to OPEN a file with the given
filename in read mode and ASSIGN the contents of the file to
`history`. IF the file is not found, PRINT an error message and
CALL the `menu` function.

2. CREATE a function `menu`:
    1. CREATE an instance of `Calculator`.
    2. WHILE True:
        1. DISPLAY a menu with options for different
operations.
        2. GET the user's choice.
        3. IF the choice is '1', GET two numbers from the user,
ADD them using the `add` method, and PRINT the result.
        4. ELSE IF the choice is '2', GET two numbers from the
user, SUBTRACT them using the `subtract` method, and PRINT the
result.
        5. ELSE IF the choice is '3', GET two numbers from the
user, MULTIPLY them using the `multiply` method, and PRINT the
result.
        6. ELSE IF the choice is '4', GET two numbers from the
user, DIVIDE them using the `divide` method, and PRINT the
result.
        7. ELSE IF the choice is '5', GET a filename from the
user, SAVE the `history` to the file using the `save_history`
method, and PRINT a success message.
        8. ELSE IF the choice is '6', GET a filename from the
user, LOAD the `history` from the file using the `load_history`
method, PRINT a success message, and PRINT the `history`.
        9. ELSE IF the choice is '7', GET a file path from the
user, REPLACE certain characters in the file content, SAVE the
corrected content to a new file, and PRINT a success message.
        10. ELSE IF the choice is '0', PRINT a message and EXIT
the loop.
        11. ELSE, PRINT an error message and EXIT the loop.

3. IF the script is run directly (not imported), CALL the
`menu` function.
```

B. TASK 2 OPERATORS AND CONTROL STRUCTURES

The code involves the use of various operators, including the arithmetic operators **+, -, *,** and / for the calculator operations, the assignment operator = to assign values to variables, and the comparison operator == to check if the second number in the division operation is zero.

The code uses ***if-elif-else*** statements to select which block of code to execute based on certain conditions. For example, in the ***divide*** method, if the second number is zero, an error message is appended to the history and the method returns None. Otherwise, the division is performed, the operation is appended to the history, and the result is returned.

The code uses a ***while*** loop to continuously display the calculator menu and perform operations based on the user's choice until the user chooses to exit. It also uses a ***for*** loop to iterate over the history list when saving the history to a file.

C. TASK 3 STRING MANIPULATION

The code manipulates strings using different operations. It uses the ***format*** method to create strings detailing the calculator operations and the results.

The code also explores formatting and parsing techniques. It uses ***f-strings***, a type of string formatting in Python, to create the strings for the calculator history. It uses the ***readlines*** method to parse the history from a file into a list of strings, and the ***strip*** method to remove trailing newline characters when printing the history.

D. TASK 4 FILE HANDLING AND EXCEPTION HANDLING

The code reads data from a file, performs operations, and writes the results back to another file. It uses the ***open*** function to open files in read or write mode, the ***read***, ***readlines***, and ***write*** methods to read from or write to files, and the ***with*** statement to automatically close the files after the operations.

The code implements exception handling to manage potential errors. It uses a ***try-except*** block to handle the ***FileNotFoundError*** that may be raised when trying to open a file that does not exist. In the ***except*** block, it prints an error message and calls the ***menu*** function to continue the program.

Typing Corrector:

A. TASK 1 ALGORITHM DESIGN AND PRINCIPLES

The provided Python code is a simple text correction tool that replaces certain characters in a text file and saves the corrected content to a new file.

The main algorithm of the code is designed to solve a specific problem: correcting certain characters in a text file. The algorithm is implemented in Python and involves the following steps:

1. Get the file path from the user.
2. Open the file and read its content.
3. Replace certain characters in the content.
4. Save the corrected content to a new file.
5. Print a success message.

The code uses basic input/output for user interaction. It uses the ***input*** function to get the file path from the user, and the ***print*** function to display the success message.

```
ALGORITHM (Pseudocode)

1. PROMPT the user to enter the file path and STORE it in
`file_path`.
2. OPEN the file at `file_path` in read mode.
3. READ the content of the file and STORE it in `content`.
4. CLOSE the file.
5. REPLACE "0" with "o" in `content`.
6. REPLACE "1" with "i" in `content`.
7. REPLACE "3" with "e" in `content`.
8. REPLACE "4" with "a" in `content`.
9. REPLACE "5" with "s" in `content`.
10. REPLACE "7" with "t" in `content`.
11. REPLACE ".txt" with "_corrected.txt" in `file_path` and
STORE it in `new_file_path`.
12. OPEN the file at `new_file_path` in write mode.
13. WRITE `content` to the file.
14. CLOSE the file.
15. PRINT "File corrected and saved as:" and `new_file_path`.
```

B. TASK 3 STRING MANIPULATION

The code manipulates strings using the ***replace*** method to correct certain characters in the text file.

C. TASK 4 FILE HANDLING AND EXCEPTION HANDLING

The code reads data from a file, performs operations, and writes the results back to another file. It uses the ***open*** function to open files in ***read*** or ***write*** mode, the read and write methods to read from or write to files, and the with statement to automatically close the files after the operations.

The code implements exception handling to manage potential errors. It uses a ***try-except*** block to handle the ***FileNotFoundError*** that may be raised when trying to open a file that does not exist. In the ***except*** block, it prints an error message.

III. OUTPUTS AND SNIPPETS

A. Snippets

```
class Calculator:
    def __init__(self):
        self.history = []

#This line of code is inside the __init__ method of a Python class. The __init__ method is a special method in Python classes, it's the constructor method which is automatically called when an object of the class is created.

#The line self.history = [] is initializing an empty list and assigning it to the instance variable self.history. This means that each object of this class will have its own history list. This list is used to store the history of the calculator, likely a history of operations or calculations that have been performed.

    def add(self, num1, num2):
        result = num1 + num2
        self.history.append(f'Added {num1} and {num2} = {result}')
        return result

#This line calculates the sum of num1 and num2 and assigns it to the variable result. The add method then appends a string detailing the
```

operation to the calculator's history and returns the result.

```
    def subtract(self, num1, num2):
        result = num1 - num2
        self.history.append(f'Subtracted {num2} from {num1} = {result}')
        return result

    def multiply(self, num1, num2):
        result = num1 * num2
        self.history.append(f'Multiplied {num1} and {num2} = {result}')
        return result

    def divide(self, num1, num2):
        if num2 == 0:
            self.history.append('Error: Division by zero is not allowed.')
            return print("Error: Division by zero is not allowed.")
        result = num1 / num2
        self.history.append(f'Divided {num1} by {num2} = {result}')
        return result

    def save_history(self, filename):
        with open(filename, 'w') as f:
            for line in self.history:
                f.write(f'{line}\n')
```

#This line of code is inside the save_history method of the Calculator class. While this is true, the method will open the file with the given filename in write mode and iterate over the history list, writing each line to the file.

```
    def load_history(self, filename):
        try:
            with open(filename, 'r') as f:
                self.history = f.readlines()
        except FileNotFoundError:
            print("History not found.")
            menu()
```

#This line of code is inside the load_history method of the Calculator class. While this is true, the method will try to open the file with the given filename in read mode and assign the contents of the file to the history list. If the file is not found, the method will print an error message and call the menu function.

```
def menu():
    calc = Calculator()
    while True:
        print("\nCalculator Menu:")
        print("0. Exit")
        print("1. Add")
        print("2. Subtract")
        print("3. Multiply")
        print("4. Divide")
```

```
print("5. Save History")
print("6. Load History")
print("\nMore Tools:")
print ("7. Jejemon Typings Corrector")
print ("\n")
choice = input("Enter your choice: ")
if choice == '1':
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = calc.add(num1, num2)
        if result is not None:
            print(f"Result: {result}")
    except ValueError:
        print("Invalid input. Please enter a number.")
```

This part of line consists of ELSE IF or ELIF function. ELIF function checks multiple expressions for TRUE and executes a block of code as soon as one of the conditions evaluates to TRUE. This line will only run after the user inputs their choice. IF their choice is 1, then the program will ask the user to input two numbers and then the program will add the two numbers and print the result. This is done by calling the add method of the Calculator class.

```
elif choice == '2':
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = calc.subtract(num1, num2)
        if result is not None:
            print(f"Result: {result}")
    except ValueError:
        print("Invalid input. Please enter a number.")
elif choice == '3':
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
        result = calc.multiply(num1, num2)
        if result is not None:
            print(f"Result: {result}")
    except ValueError:
        print("Invalid input. Please enter a number.")
elif choice == '4':
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
```

```
                result = calc.divide(num1, num2)
                if result is not None:
                    print(f"Result: {result}")
            except ValueError:
                print("Invalid input. Please enter a number.")
        elif choice == '5':
            filename = input("Enter filename to save history: ")
            calc.save_history(filename)
            print("History saved successfully.")
        elif choice == '6':
            filename = input("Enter filename to load history: ")
            calc.load_history(filename)
            print("History loaded successfully.")
            for line in calc.history:
                print(line.strip())
        elif choice == '7':
            file_path = input("Enter the file path: ")
            try:
                with open(file_path, 'r') as file:
                    content = file.read()
            except FileNotFoundError:
                print("File not found.")
                menu()
            content = content.replace("0", "o")
            content = content.replace("1", "i")
            content = content.replace("3", "e")
            content = content.replace("4", "a")
            content = content.replace("5", "s")
            content = content.replace("7", "t")
            new_file_path = file_path.replace(".txt", "_corrected.txt")
            with open(new_file_path, 'w') as file:
                file.write(content)
            print("File corrected and saved as:", new_file_path)
```

#This line of code is inside the elif choice == '7' block. While this is true, the program will ask the user to enter a file path and try to open the file in read mode. If the file is not found, the program will print an error message and call the menu function. If the file is found, the program will read the contents of the file and replace all instances of 0 with o, 1 with i, 3 with e, 4 with a, 5 with s, and 7 with t. The program will then save the corrected content to a new file with the same name as the original file, but with _corrected appended to the end of the filename.

```
        elif choice == '0':
            print("Exiting the calculator.")
            break
        else:
            print("Invalid choice. Please enter a valid choice.")
```

```
            break

if __name__ == "__main__":
    menu()
```

B. Outputs

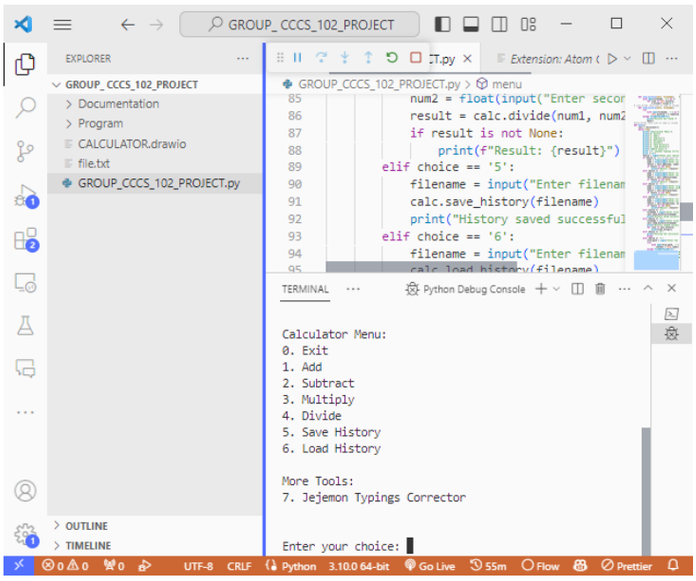


Figure 1. Program Running.

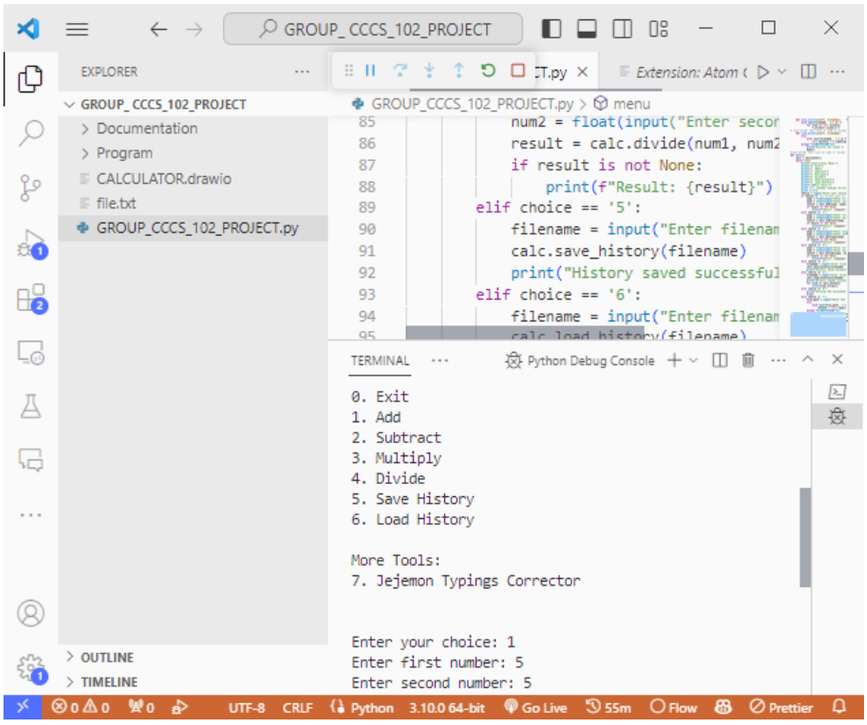
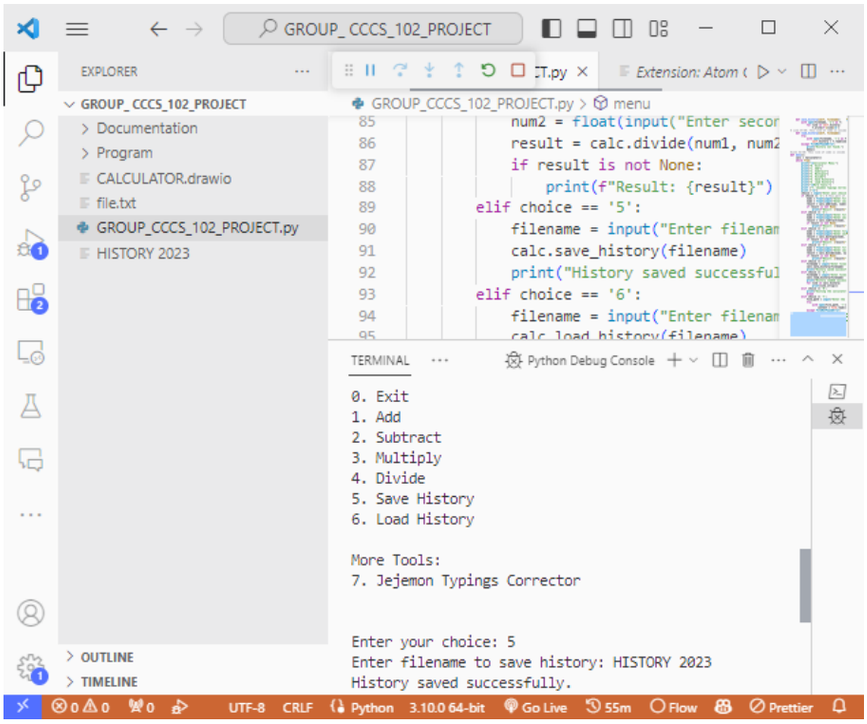


Figure 2. Basic user input and output. *The algorithm is functioning correctly and without errors.*



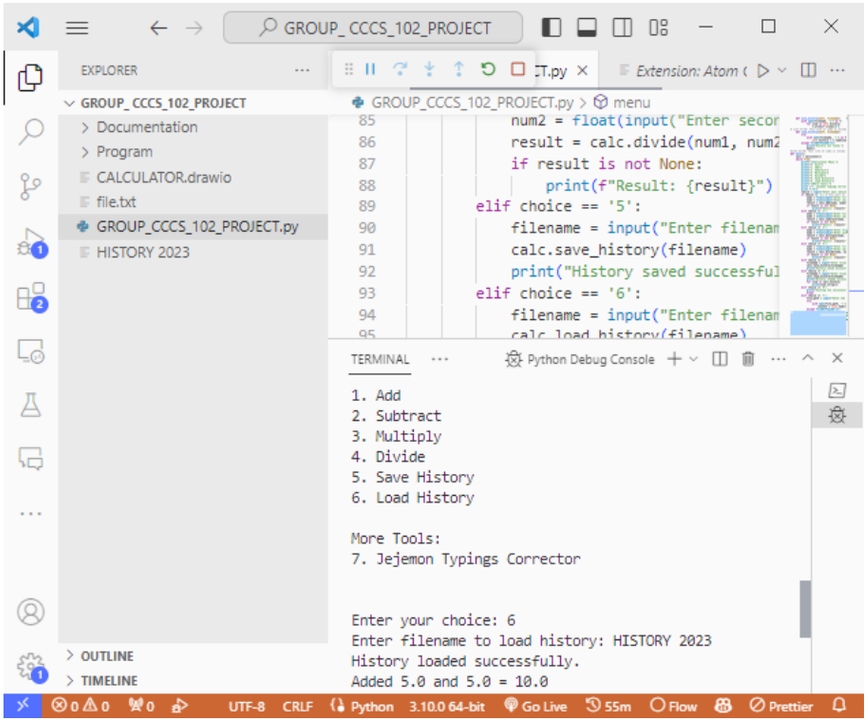


Figure 3-4. File Handling. *The program creates a text file in the same directory as the running program. When the program performs Load History, it reads the text file created by the program in the same directory as the running program.*

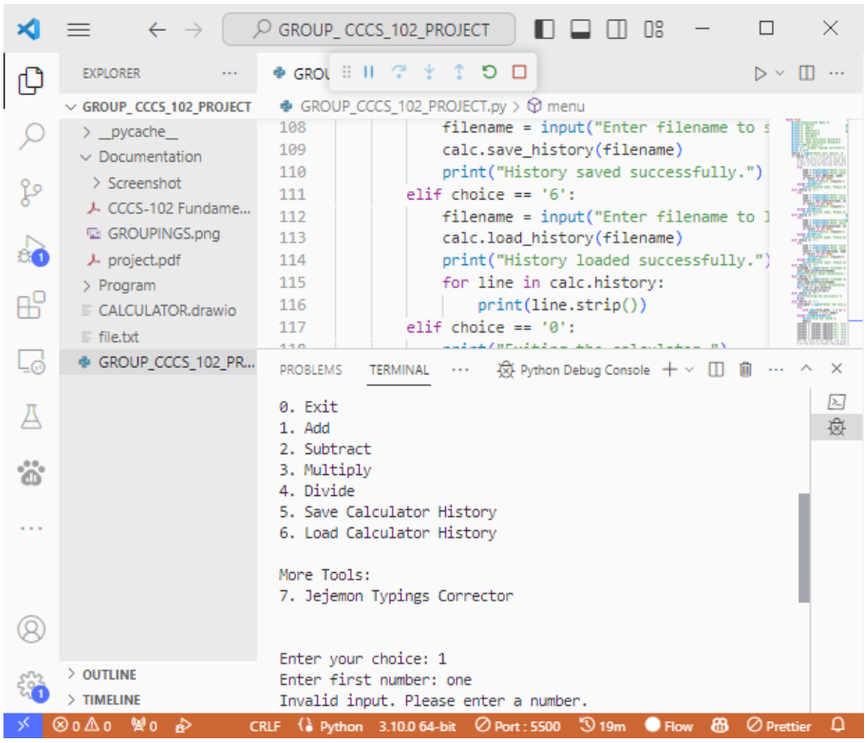
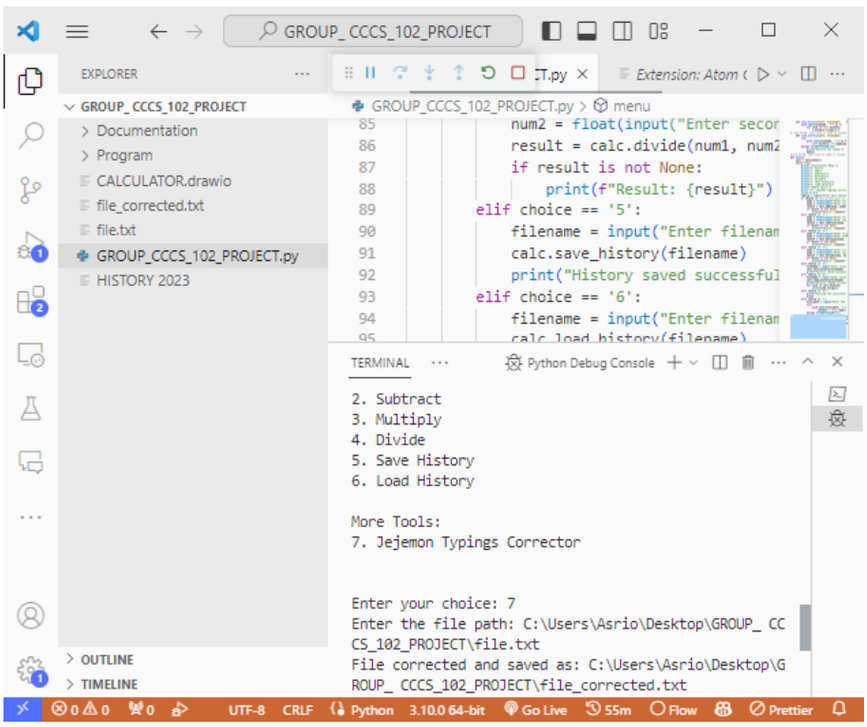


Figure 5. Exception Handling. *The program outputs an error message when the user inputs an invalid text. For example in add method.*



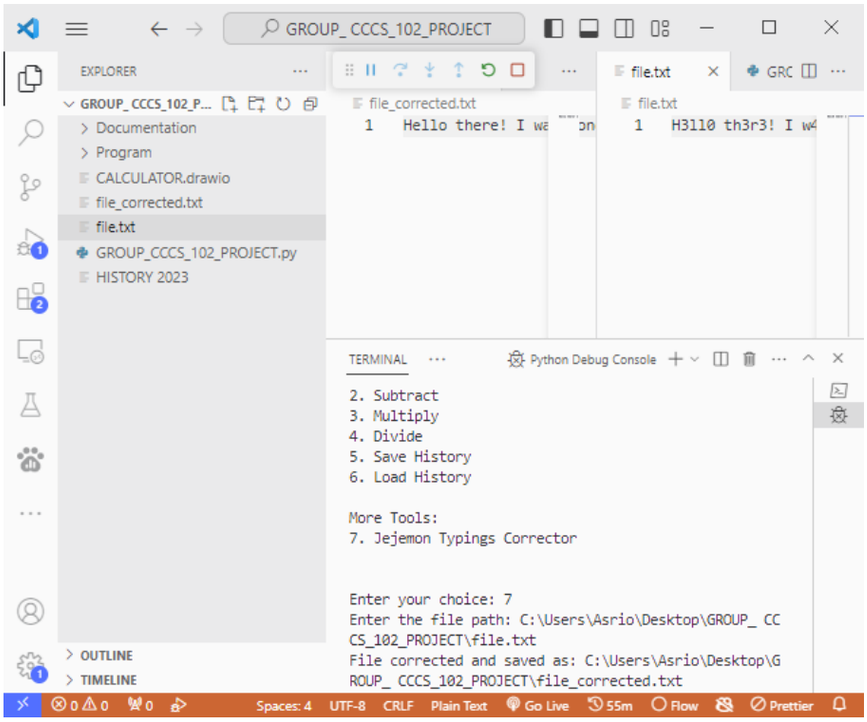


Figure 5-6. Jejemon Typings Corrector. *The program asks for the location of the file, reads it, and replaces all the characters that are assigned in the code; then it creates a new file with the "_corrected" name.*

IV. CHALLENGES

File Handling

One of the main challenges in creating this project was the file handling task. File handling in Python involves opening a file, reading or writing data, and then closing the file. This is a complex task that the members have faced during the process, especially when dealing with different modes of opening a file and handling potential errors.

Flowcharting

One of the members was assigned to create a flowchart for the project, and this was a challenge for them. A flowchart is a visual representation of an algorithm or a

process, showing the steps as boxes of various kinds and their order by connecting them with arrows.

“So-so” Programming Background

This challenge during the creation process makes understanding and implementing complex concepts like file handling and control structures more difficult. On the other hand, this is overcome by additional learning resources and practicing Python programming.

V. CONCLUSIONS AND KEY TAKEAWAYS

A. CONCLUSIONS

The code is a good example of a simple Python application that combines various concepts such as algorithm design, string manipulation, file handling, and exception handling. It demonstrates how these concepts can be used together to solve a specific problem.

Despite the challenges faced during the creation of the code, such as a "so-so" programming background and difficulties with file handling and flowchart creation, the team was able to overcome these challenges through learning resources, practice, and hands-on experience. This highlights the importance of continuous learning and practice in software development.

B. KEY TAKEAWAYS

- Design an algorithm first before coding.
- Take time to understand the problem and requirements before starting to code.
- Use clear and descriptive variable names to improve code readability.
- Test the code thoroughly to ensure it works as intended.

- Continuously learn and improve coding skills through resources and practice.